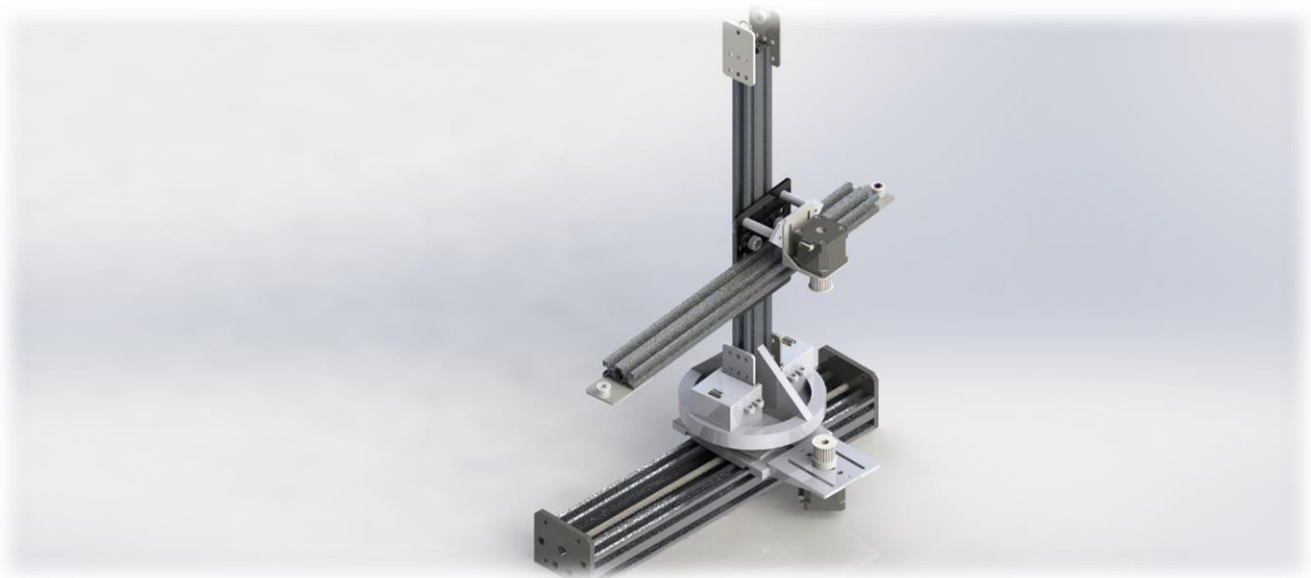




Robotics Engineering Course Project Report

Hybrid Cartesian–Cylindrical Robot with 2D drawing feature



Presented by:

Malek Sameh Ismail 1220271

Ahmed Osama Shehata 1220029

Mohamed Ashraf 4230168

Abdelrahman Samir 1230056

Group 21

Supervised by:

Dr. Hossam H. Ammar

Eng. Miriam Hani

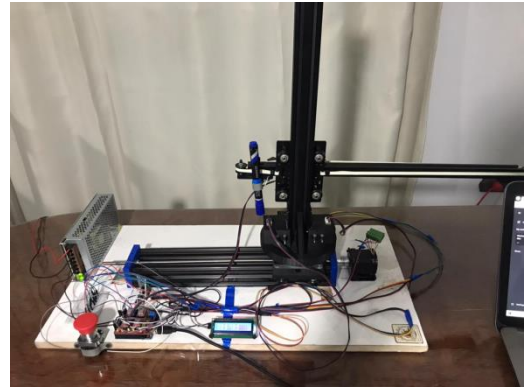
Contents

1. Introduction.....	4
2. Design Overview	5
2.1. Robot Configuration & Design Rationale	5
2.2. Mechanical Design Concept	5
2.3. Electrical System Integration.....	5
2.4. Software Architecture	6
2.4.1. High-Level Controller (The Master)	6
2.4.2. Low-Level Controller (The Slave)	7
2.4.3. Communication & Synchronization Protocol.....	7
2.4.4. Safety Layer Implementation	7
3. Kinematic & Workspace Analysis	8
3.1 Degrees of Freedom (DOF).....	8
3.2 Kinematic Analysis	8
3.3 Calculations of Mass & Forces.....	12
3.4 Workspace Analysis	17
4. Simulation Model & Motion Planning.....	18
4.1 URDF Modeling & Physical Definition	18
4.2 Simulation Environment (Gazebo & ROS Control)	19
4.3 Motion Planning & G-Code Execution.....	19
4.4 Visualization & Validation (Rviz).....	19
5. Hardware Fabrication and Assembly.....	20
5.1. Fabrication of Components	20
5.2. Assembly Process	20
5.3. Physical Alignment & Mechanical Calibration.....	20
5.3.1. Instability of carriage V-Wheels.....	20
5.3.2. Rotating base teeth problem	21
6. Control Architecture & Algorithm	22
6.1 Control System Overview	22

6.2 Embedded Firmware Logic (Arduino).....	22
6.3 Inverse Kinematics (IK) Algorithm	23
6.4 G-Code Interpreter & Drawing Logic.....	24
6.5 Operating Modes.....	24
7. Calibration Method & Results	25
7.1 Homing Sequence & Coordinate Zeroing	25
7.2 Z-Axis (Pen Height) Tuning	25
7.3 Step-per-Unit Verification	26
7.4 Final Calibration Parameters	27
7.5 Repeatability Test.....	27
8. Testing, Performance Evaluation & Error Analysis.....	27
8.1 Demonstration Task Overview	27
8.1.1. Arduino Firmware	27
8.1.2. Python Control Program.....	27
8.2 Initial Performance and Identified Issues.....	28
8.3 Corrective Actions and Performance Improvement	28
8.4 Quantitative Error Analysis.....	30
8.5 Conclusion of Testing Phase	30
9. Demonstration Description & Results	30
9.1 Software Control Logic	30
9.2 Demonstration Tasks & Results.....	31
9.3 Safety & Reliability Verification.....	32
10. Optional Enhancements:	32
11. Conclusion & Future Work	33
11.1 Conclusion	33
11.2 Future Work	33
12. References	34

1. Introduction

In the rapidly evolving world of industrial automation, robotic manipulators form the backbone of modern manufacturing systems, enabling applications ranging from high-precision assembly to heavy material handling. Traditionally, engineers have been required to choose between two distinct robotic architectures: **Cartesian gantry systems**, which offer exceptional rigidity and accuracy but occupy a large physical footprint, and **cylindrical or SCARA robots**, which provide greater agility and rotational flexibility at the cost of reduced stiffness, particularly at extended reach. This project was motivated by the need to bridge this gap through the development of a robotic system that combines the structural stability of linear gantries with the adaptability of articulated mechanisms.



To achieve this objective, a **hybrid Cartesian–cylindrical robotic manipulator** was designed and fabricated using a **P–R–P–P (Prismatic–Revolute–Prismatic–Prismatic)** configuration. Unlike conventional designs that rely exclusively on either linear or rotational motion, the proposed system employs a robust linear base rail that serves as the robot’s backbone, supporting a rotating platform and an extending arm assembly. This hybrid configuration enables coverage of a wide operational workspace while minimizing wasted floor space, effectively combining long-range translational motion with localized rotational dexterity.

Beyond mechanical design, the project aimed to evaluate the system’s ability to execute **precise, coordinated motion**. To rigorously validate the manipulator’s performance, the robot was assigned the task of **automated 2D drawing and plotting** (more about the in **section 6**). This application was intentionally selected due to its sensitivity to positional errors, where even small deviations become immediately visible in the drawn output. Accurately tracing vector-based shapes requires precise synchronization between all joints: linear translation along the base, rotation about the vertical axis, and coordinated motion of the vertical and horizontal extension axes. Consequently, producing a straight line or smooth curve in Cartesian space corresponds to a complex, nonlinear motion profile in joint space, necessitating real-time **inverse kinematics** computation.

To manage this level of complexity, a **master–slave control architecture** was implemented. A high-level Python-based planner performs image parsing, trajectory generation, and kinematic calculations, while an embedded Arduino controller executes the resulting motor commands with precise timing. This report presents the complete engineering process, encompassing kinematic modeling, simulation, mechanical fabrication, assembly, and software integration. The successful execution of the drawing task ultimately demonstrates that the proposed hybrid PRPP architecture is capable of achieving the accuracy and smoothness required for drafting, plotting, and other precision manipulation applications.

2. Design Overview

2.1. Robot Configuration & Design Rationale

The developed manipulator is a **four-degree-of-freedom (4-DOF) PRPP robotic system**. This serial-chain configuration consists of:

- A **Prismatic joint** providing linear motion along the **X-axis**,
- A **Revolute joint** enabling rotation about the **Z-axis**,
- A second **Prismatic joint** moving along the **Z-axis**, and
- A final **Prismatic joint** translating along the **Y-axis**.

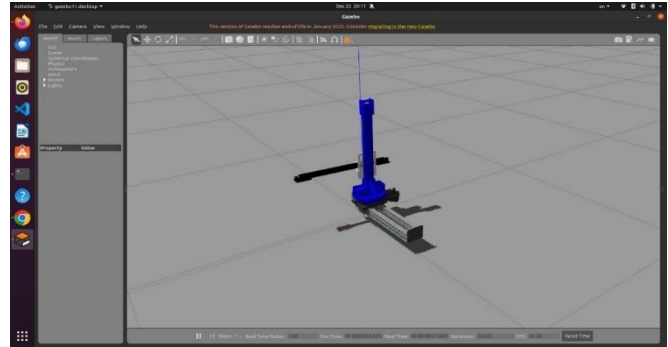


Figure 1 : Our PRPP Robot viewed from Gazebo

This specific joint arrangement—where the revolute joint is positioned between two prismatic axes—introduces a key functional advantage. It allows the entire vertical (Z) and horizontal (Y) extension assembly to **yaw about the z-axis**, enabling the end effector to sweep from one side of the robot to the other. As a result, the robot achieves a significantly expanded and more flexible workspace compared to a conventional Cartesian (PPP) configuration. Further details on workspace improvement are presented in the **Workspace Analysis** section.

2.2. Mechanical Design Concept

The mechanical design prioritizes **simplicity, modularity, and manufacturability**, relying heavily on standardized linear motion components and aluminum profiles. This approach minimizes fabrication complexity while maintaining sufficient rigidity and repeatability for accurate robotic manipulation. Each joint was designed as an independent mechanical module, facilitating straightforward assembly, alignment, and future modifications. More on this and the challenges that faced us in **section 5**

2.3. Electrical System Integration

The electrical architecture is built around a centralized controller managing five actuators.

- **Actuators:** Five **NEMA 17 Stepper Motors** are used to drive the axes.
- **Power Distribution:** A switched-mode power supply (SMPS) converts **220V AC to 12V DC** to power the motor drivers and logic boards.

- **Controller Interface:** An **Arduino Uno** equipped with a **CNC Shield** is used to organize wiring and house the drivers. This eliminates "rat's nest" wiring and ensures reliable connections.

Dual-Motor Series Configuration (Z-Axis): The Z-axis (Lift) utilizes a dual-motor setup to ensure stability and sufficient lifting force. However, the system uses only **4 Stepper Drivers** for the 5 motors.

- **Implementation:** The two Z-axis motors are wired in **series** to a single driver.
- **Electrical Justification:** In a series connection, the current remains constant through both coils, ensuring identical torque output from both motors without exceeding the current limit of the single driver. The voltage is divided between the two motors, which is handled efficiently by the 12V supply.

Safety System: A physical Emergency Stop (E-Stop) button is integrated into the control loop.

- **Logic:** The button is wired in a **Normally Closed (NC)** configuration. This is a fail-safe design; if the wire is cut or the button is pressed, the circuit opens (Logic HIGH), triggering the firmware's interrupt routine to immediately halt all motion.

2.4. Software Architecture

The control system is architected on a **Master-Slave topology**, designed to decouple the computationally intensive trajectory planning from the real-time signal generation required for smooth motor control.

2.4.1. High-Level Controller (The Master)

The "Master" node is a Python-based application running on a host PC. It serves as the robot's high-level planner and user interface.

- **Trajectory Generation:** The system parses standard G-code files, extracting Cartesian coordinates (X, Y, Z). A pre-processing algorithm scans the file boundaries and calculates a dynamic Scaling Factor to map large images into the robot's physical workspace limits.
- **Inverse Kinematics (IK) Engine:** A custom solver translates desired Cartesian positions into Joint Space parameters. This engine handles the hybrid redundancy of the P-R-P-P structure by prioritizing the Base Rail (X) for large displacements and utilizing the Extension Arm (Y) for local manipulation. It also applies "Smart Rotation" logic to minimize angular travel).
- **Data Serialization:** Coordinates are converted from Centimeters (Python logic) to Millimeters (Arduino driver), formatted into ASCII packets, and transmitted via USB Serial.

2.4.2. Low-Level Controller (The Slave)

The "Slave" node is an Arduino Uno running custom firmware, acting as the real-time motion driver.

- **Pulse Generation:** The firmware utilizes the AccelStepper library to generate precise square waves for the stepper drivers. It manages acceleration ramps and maximum velocity constraints to prevent motor stalling under load.
- **Non-Blocking Loop:** The main execution loop is designed to be non-blocking. It continuously polls the serial buffer for new commands and monitors the Emergency Stop pin (A0) while simultaneously toggling step pins.
- **Hardware Abstraction:** The firmware maps the logical axes to the physical CNC Shield pinout, handling the series-wired Z-axis motors as a single logical unit.

2.4.3. Communication & Synchronization Protocol

To bridge the asynchronous nature of a PC (fast, non-deterministic) and a microcontroller (slow, deterministic), a robust handshake protocol was implemented over UART (115200 Baud).

- **The "Wait" Flag (W1):** Standard buffering causes command overflow in drawing applications. To solve this, drawing commands are tagged with a W1 flag.
- **Handshake Logic:** When the Arduino receives a W1 tagged command, it enters a specialized blocking mode. It executes the motion completely and, upon reaching the target, transmits a DONE signal back to the PC.
- **Flow Control:** The Python script waits for this DONE token before sending the next line of G-code. This ensures perfectly sharp corners and prevents the loss of data during complex vector plotting.

2.4.4. Safety Layer Implementation

Safety is enforced hierarchically across both software layers:

1. **Logic Safety (Python):** The IK solver includes a clamping mechanism. If a target coordinate exceeds the defined RAIL_MAX_X or ARM_MAX_Y, the software caps the value or rejects the move before transmission, preventing frame collisions.
2. **Physical Safety (Arduino):** A hardware interrupt is linked to the Emergency Stop button (Normally Closed). If the circuit breaks, the firmware immediately halts all step generation and locks the system state until a manual reset is performed.

3. Kinematic & Workspace Analysis

3.1 Degrees of Freedom (DOF)

DOF Identification

Joint Type	Motion Type	Axis	DOF Contribution
Revolute (Base)	Rotation	θ	1
Prismatic (X-axis)	Linear	X	1
Prismatic (Y-axis)	Linear	Y	1
Prismatic (Z-axis)	Linear	Z	1

DOF Calculation

Component	Symbol	Quantity
Total links (including base)	(n)	5
Total joints	(j)	4
DOF per joint	(fi)	1

$$\text{DOF} = 6(n - 1 - j) + \sum f_i$$

$$\text{DOF} = 6(5 - 1 - 4) + 4$$

$$\text{DOF} = 6(0) + 4$$

$$\boxed{\text{DOF} = 4}$$

3.2 Kinematic Analysis

DH Table:

#	d	θ	α	a
0-1	x	0	90	0
1-2	z	Θ_z	0	0
2-3	y	-90	90	0
3-end effector	0	0	90	0

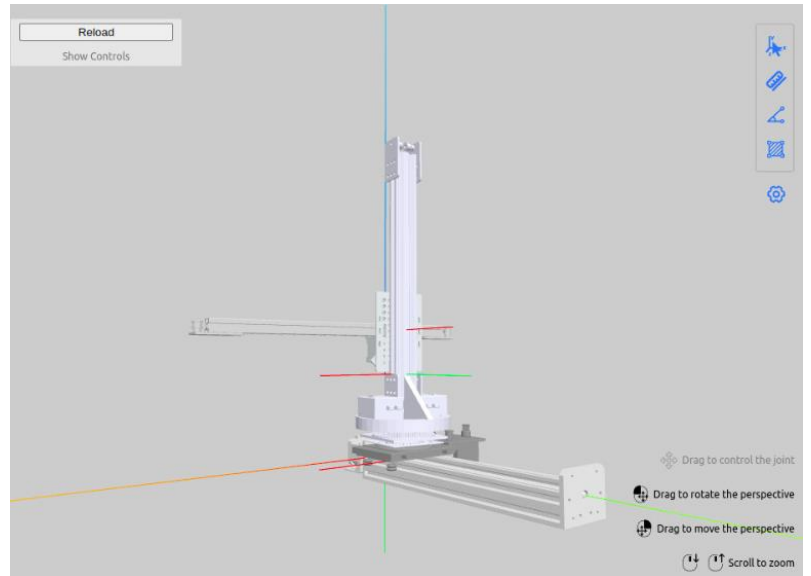


Figure 2 : DH Table Reference Frames

Standard DH Transformation Reminder can be found in Appendix 1

All unspecified parameters are **zero**.

Definition of Joint Variables

- x
Linear displacement of the **first prismatic joint**, representing translation of the robot base carriage along the **global X-axis**. This motion provides long-range horizontal positioning of the entire manipulator.
- θ
Angular displacement of the **revolute joint**, defining the yaw rotation of the manipulator about the **Z-axis**. This rotation allows the end effector to sweep laterally across the workspace.
- z
Linear displacement of the **third prismatic joint**, representing vertical translation of the end effector along the **global Z-axis**. This motion enables height adjustment independent of the robot's orientation. A part of it is a constant offset from the θ reference frame (14 cm)
- y
Linear displacement of the **second prismatic joint**, corresponding to the radial extension of the arm along the **local Y-axis** after the yaw rotation. This variable controls the horizontal reach of the end effector relative to the rotating base. A part of it is a constant offset from the z reference frame (3 cm)

Note that all linear axes range from 0 to 400mm.

Base → First Link

Given:

- $d = x$
- $\alpha = 90^\circ$

$${}^0T_1 = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & x \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

First Link → Second Link

Given:

- $d = z$
- $\theta = \theta_z$

$${}^1T_2 = \begin{bmatrix} \cos \theta_Z & -\sin \theta_Z & 0 & 0 \\ \sin \theta_Z & \cos \theta_Z & 0 & 0 \\ 0 & 0 & 1 & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Second Link → Third Link

Given:

- $d = y$
- $\theta = -90^\circ$
- $\alpha = 90^\circ$

Using

$$\cos(-90) = 0, \sin(-90) = -1$$

$${}^2T_3 = \begin{bmatrix} 0 & 0 & -1 & 0 \\ -1 & 0 & 0 & 0 \\ 0 & 1 & 0 & y \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Third Link → End Effector

Given:

- $\alpha = 90^\circ$

$${}^3T_E = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Total Transformation Matrix

$${}^0T_E = {}^0T_1 * {}^1T_2 * {}^2T_3 * {}^3T_E$$

Step-by-step multiplication result

$${}^0T_E = \begin{bmatrix} -\sin \theta_Z & 0 & -\cos \theta_Z & -y \cos \theta_Z \\ -\cos \theta_Z & 0 & \sin \theta_Z & y \sin \theta_Z \\ 0 & 1 & 0 & x + z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Resulting End-Effector Position

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = \begin{bmatrix} -y \cos \theta_Z \\ y \sin \theta_Z \\ x + z \end{bmatrix} \quad \text{Not so logical (look below)}$$

Why this result is physically wrong

Even though the math is correct **for the given DH parameters**, the result violates the robot's geometry:

- x incorrectly contributes to z
- No term gives $X = x + y \cos \theta$
- Excessive $\pm 90^\circ$ twists artificially rotate axes

This confirms that the **error is in the DH frame assignment**, not the multiplication.

Although the forward kinematics were derived correctly from the assumed DH parameters, the resulting end-effector position revealed non-physical axis coupling. This discrepancy highlights an improper assignment of link twists and frame orientations rather than an algebraic error in the transformation multiplication

Let's consider another approach.

Individual Transformation Matrices

Joint 1 (Prismatic X)

$${}^0T_1 = T_z(x) = \begin{bmatrix} 1 & 0 & 0 & x \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Joint 2 (Revolute Z)

$${}^1T_2 = R_z(\theta) = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & 0 \\ \sin \theta & \cos \theta & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Joint 3 (Prismatic Y, local frame)

$${}^2T_3 = T_y(y) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & y \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Joint 4 (Prismatic Z)

$${}^3T_4 = T_z(z) = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Total Transformation Matrix

$${}^0T_E = {}^0T_1 {}^1T_2 {}^2T_3 {}^3T_4$$

After multiplication:

$${}^0T_E = \begin{bmatrix} \cos \theta & -\sin \theta & 0 & x + y \cos \theta \\ \sin \theta & \cos \theta & 0 & y \sin \theta \\ 0 & 0 & 1 & z \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

Final End-Effector Position (Correct)

$$\begin{array}{lcl} X & = & x + y \cos \theta \\ Y & = & y \sin \theta \\ Z & = & z \end{array}$$

And this seems more logical

3.3 Calculations of Mass & Forces

Mass Calculations

Part (as listed)	Typical mass (per unit)	Notes / source
20 × 40 aluminium extrusion (per meter)	~0.79 – 0.85 kg / m.	Vendor specs for standard 2020×40 profiles. (tnutz.com)
20 × 20 aluminium extrusion (per meter)	~0.43 – 0.48 kg / m.	Common manufacturers list ≈0.45–0.48 kg/m. (item24.com)
Lead screw (typical T8 / 8 mm trapezoidal / stainless) (per meter)	~0.31 kg / m (linear mass density).	Product datasheets list ~0.31 kg/m for 8 mm lead screws. (systeal.com)
NEMA 17 stepper motor (typical 42×42×~48–60 mm packs)	~200 – 350 g (common example: ≈288 g).	Many common NEMA17s weigh about 0.25–0.35 kg; example spec shows 288 g. (Robu)
GT2 timing pulley (20T, aluminium, small bores)	~4 – 15 g (typical 20T pulleys ≈4–15 g; many common 20T ≈~5–15 g).	Product pages list weights from ~4 g up to ~15 g depending bore/flange. (cncegy.com)
GT2 idler pulley / tiny timing idler	~3 – 6 g (small aluminium/steel idlers often	Small idlers commonly listed ~3.7 g or ~5.7 g in vendor

Link	Parts	Mass
Linear Base	40cm Aluminum Profile + 50cm Lead screw + 3d printed end plates + V slot gantry plate (0.26kg) + NEMA 17 (0.27kg)	1.1 kg
Rotating Base	3d printed susan bearing mount + 4" susan bearing + 3d printed rotating base + NEMA 17 + thrust bearing 32*46mm	0.59 kg
Vertical Link	20*40 aluminum profile + 2 NEMA 17 + 2 straight motor plates + 2 gt2 pulley + 2 gt2 idler + 2 pulley plates + V slot gantry plate	1.26 kg
Horizontal	20*20 aluminum profile + custom made motor plate + L	0.58 kg

	~3–6 g).	listings. (Bastelgarage)
V-slot / V-slot gantry plate (small aluminium plate, e.g. 100×88 mm)	~ 50–70 g (example: 54 g for a 100×88 mm plate).	Vendor spec: 100×88 mm gantry plate ≈54 g. Larger plates scale up. (systeal.com)
Lazy-Susan / “susan” bearing (small 3" style)	~ 80 – 180 g (example: a pack/set of two 3" bearings listed as 0.18 kg total → ~90 g each).	Small turntable bearings often listed ~0.09–0.18 kg per piece/set depending on size/material. (Amazon)
3D-printed small parts (end plates, motor plates, rotating base, etc.)	very variable — typical small plates ~ 5–80 g depending on size, thickness and infill.	3D printed mass depends on part volume & filament density (PLA ≈ 1.24 g/cm³). See method below to compute from STL/volume.

shaped motor plate + NEMA 17 + 2 idler pulleys + 2 pulley plates
--

Force Analysis

Nice — below is the same full write-up but **with efficiency ignored** in all calculations (i.e., treat the formulas as ideal). I'll show every arithmetic step so you can follow / verify each number.

General notes & assumptions (updated)

- Each NEMA-17 holding torque $= 0.45 \text{ N}\cdot\text{m}$.
- Lead screw: T8, **lead** = **2.0 mm** = **0.002 m** (2 mm linear advance per motor rev).
- GT2 pulley effective radius $r = 6.37 \text{ mm} = 0.00637 \text{ m}$.
- Rotating base payload radius $R = 0.05 \text{ m}$.
- $g = 9.81 \text{ m/s}^2$.
- Safety factor (recommended) = 3 (I'll report raw and safe values).

Useful formulas (no efficiency factors)

Lead-screw (ideal): $F_{\text{ideal}} = \frac{2\pi T}{\text{lead}}, m = \frac{F_{\text{ideal}}}{g}$

Pulley / belt (direct drive): $F = \frac{T}{r}, m = \frac{F}{g}$

Rotary (turntable) at radius R: $T = m g R \Rightarrow m = \frac{T}{gR}$

1) Lead-screw linear force (ideal)

Compute $\frac{2\pi T}{\text{lead}}$:

- $2\pi \cdot 0.45 = 2.83 \text{ N}\cdot\text{m}$

Divide by lead (0.002 m):

- $F_{\text{ideal}} = 2.83/0.002$

- $F_{\text{ideal}} = 1413.72 \text{ N}$

Convert to mass:

- $m = 1413.716694115407/9.81$
- $m = 144.1140672484148 \text{ kg} \rightarrow \approx 144.11 \text{ kg of load to be translated}$

2) Pulley (r = 0.00637 m)

Compute linear force:

- $F = 0.45/0.00637$
- $F = 70.64819220257231 \text{ N} \rightarrow \approx 70.65 \text{ N}$

So, the motor produces a linear force of $\approx 70.65 \text{ N}$ along the belt.

Convert to mass:

- $m = 70.64819220257231/9.81$
- $m = 7.205158359343238 \text{ kg} \rightarrow \approx 7.21 \text{ kg (assuming no belt failure)}$

3) Rotational mass at R = 0.05 m

$$m = \frac{T}{gR} = \frac{0.45}{9.81 \times 0.05}$$

$$m = 0.45/0.4905 = 0.9175 \text{ kg}$$

Per-link results (raw theoretical maxima — static)

Link	Motor configuration	Raw max load (kg) — static
Linear Base	1 × NEMA-17 on T8 lead screw (2 mm)	$\approx 144.11 \text{ kg}$
Rotating Base	1 × NEMA-17 directly driving table radius $R = 0.05\text{m}$	$\approx 0.9175 \text{ kg}$
Vertical Link	2 × NEMA-17 (interpretation options)	Single motor case: $\approx 144.11 \text{ kg}$ (no coupling between the motors)

Horizontal	1 × NEMA-17 driving GT2 pulley $r = 0.00637\text{m}$	$\approx 7.21\text{ kg}$

Conservative / recommended payloads (apply safety factor = 3)

Divide raw maxima by 3:

Link	Raw max (kg)	Recommended safe payload $\approx$ (raw / 3)
Linear Base	144.11 kg	$\approx 48.04\text{ kg}$
Rotating Base (R=5 cm)	0.9175 kg	$\approx 0.306\text{ kg}$
Vertical Link	144.11 kg (single) $\rightarrow$ 288.23 kg (two motors)	$\approx 48.04\text{ kg}$ (single) $\rightarrow \approx 96.08\text{ kg}$ (two motors)
Horizontal (belt)	7.21 kg	$\approx 2.40\text{ kg}$

Quick sensitivity notes (linear scaling)

- If motor torque T changes, all results scale linearly with T .
Example: if $T = 0.3\text{ Nm}$.
- multiply each mass above by $0.30/0.45 = 2/3$.
- If the rotational radius R doubles, allowable mass halves (since $m \propto 1/R$).

Practical caveats (important)

- **These numbers are optimistic** because ignoring real mechanical efficiency and friction removes the losses that reduce available force/torque. In practice the lead screw will have far less than 100% efficiency and motors have lower continuous torque than stall torque.
- **Acceleration & dynamics:** moving a payload requires extra torque for acceleration $m \cdot a$. Fast moves reduce achievable payload.
- **Backlash, belt stretch, bearing limits, gearbox ratings, and motor continuous ratings** may all reduce usable payload well below the “raw” numbers.

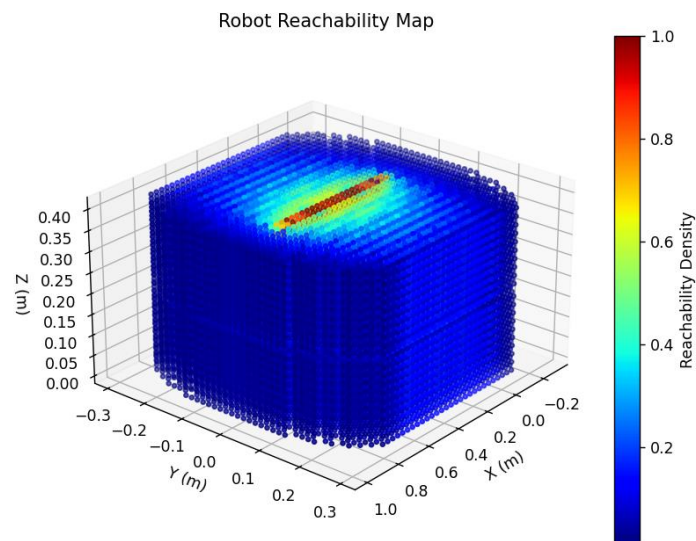
3.4 Workspace Analysis

The reachable workspace of the robot is not a simple cylinder or cube, but a **Prismatic-Cylindrical Volume** (often described as a "stadium" or "capsule" shape extruded vertically).

Workspace Dimensions:

- **X-Range:** 0 to 24.0 cm (Limited by Rail).
- **Y-Range (Reach):** 3.8 cm to 33.8 cm (Limited by Arm Extension).
- **Z-Range:** 0 to 13.0 cm (Vertical Lift).

Dead Zones: Analysis reveals a **Circular Dead Zone** near the pivot point. Because the arm has a physical minimum retracted length ($ARM_MIN_Y = 3.8\text{ cm}$), the robot cannot reach coordinates inside a 3.8 cm radius from the current center of rotation. However, because the center of rotation itself moves along the X-rail, this dead zone travels with the base.



But this photo doesn't show the whole dead zones as it is real robots, these edges are

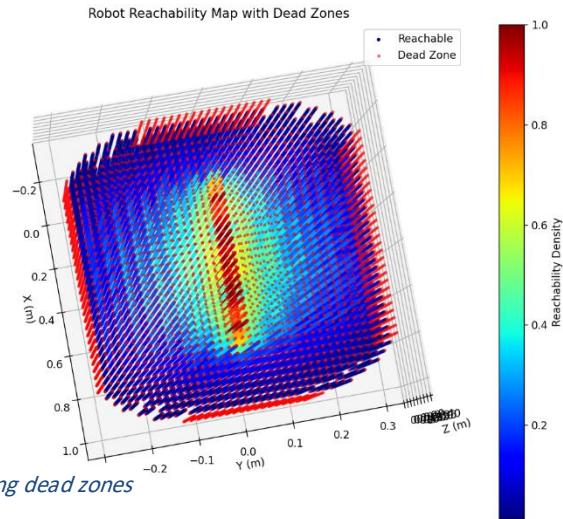


Figure 3: Illustration of the surrounding dead zones

The analysis confirms that the workspace is sufficient to cover a standard A4 paper size, provided the paper is positioned with a Y-offset of 13.0 cm from the rail, as defined in the software configuration.

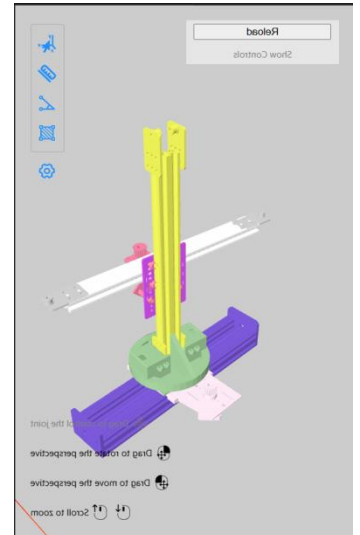
4. Simulation Model & Motion Planning

To validate the design before manufacturing, we developed a "Digital Twin" of the robot using the **ROS (Robot Operating System)** middleware and the **Gazebo** physics engine. This allowed us to verify not just the geometry, but also the dynamic behavior of the joints under gravity and load.

4.1 URDF Modeling & Physical Definition

The foundation of the simulation is the **Universal Robot Description Format (URDF)** file, which was generated from our SolidWorks CAD assembly and refined manually in XML. The URDF defines the physical properties of every component, including accurate **Inertial Properties** (Mass and Inertia Tensors). For instance, the `BaseLink` is defined with a specific mass (0.99 kg), ensuring the simulation engine calculates the correct force required to move it.

The file also defines the kinematic chain and transmission interfaces. Prismatic joints for the Rail (X), Arm (Y), and Lift (Z) axes are constrained with physical limits (e.g., \$0.25m\$ for the rail), while `<transmission>` tags using the `EffortJointInterface` allow



Gazebo to treat these joints as motor-driven axes rather than passive hinges.

4.2 Simulation Environment (Gazebo & ROS Control)

The simulation is orchestrated by a launch file that spawns the robot into an empty world and loads the necessary control plugins. We utilized **Gazebo 11** to simulate gravity and friction, which was critical for testing the Z-axis (Lift) to ensure the motor torque was sufficient to hold the "pen" up against gravity without drifting. A

`robot_state_publisher` node broadcasts the position of every link to the system at 50 Hz, while standard PID position controllers drive the joints to their target angles.

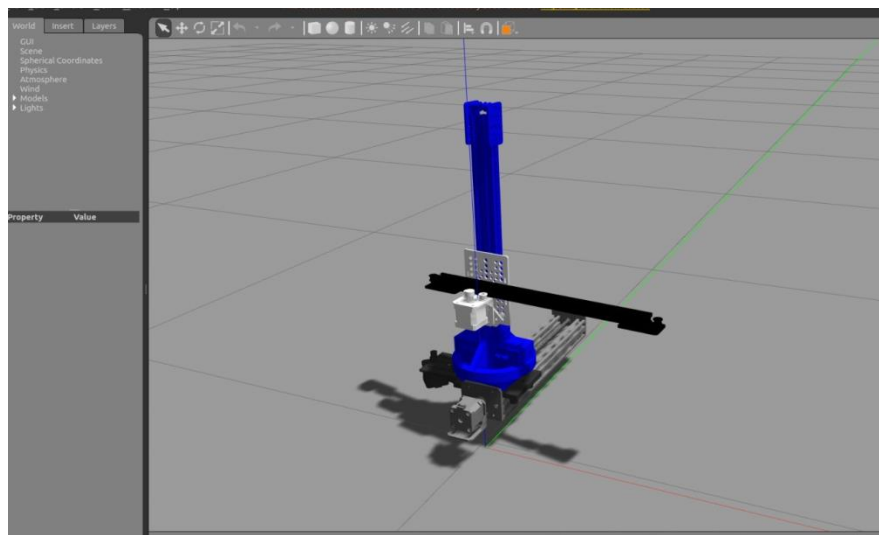
4.3 Motion Planning & G-Code Execution

The "brain" of the simulation is a custom Python node (`gcode_node.py`) that bridges the gap between static drawing files and dynamic robot motion.

1. **G-Code Parsing:** The node reads a standard `drawing.gcode` file, filtering for G1 (Linear Move) commands to extract target coordinates.
2. **Unit Conversion & Scaling:** Since ROS uses meters and G-code uses millimeters, the script automatically applies a scaling factor (1/1000) to all incoming coordinates.
3. **Trajectory Interpolation:** The node runs at a fixed loop rate of **50Hz**. It continuously publishes target commands to the four controller topics, ensuring smooth motion between waypoints.

4.4 Visualization & Validation (Rviz)

Since the Gazebo simulator does not support physical "ink," we created a virtual visualization tool using **Rviz**.. Whenever the robot executes a "Pen Down" move ($Z=0.00$), the script records the end-effector position and connects these points with a red line strip in 3D space. This provided immediate visual feedback, allowing us to confirm the kinematic accuracy of our code before physical testing.



5. Hardware Fabrication and Assembly

5.1. Fabrication of Components

Fabrication of the structural links and joints was facilitated by the use of commercially available, standardized components. Primary structural elements, including linear rails, aluminum profiles, and joint housings, were sourced directly, simplifying the procurement process.

The principal fabrication challenge resided in the **rotating base assembly (the R joint)**. This critical component required the integration of a robust thrust bearing to handle axial loads from the overhanging arm. The rotational drive was achieved via a **belt and pulley system**: a stepper motor drives a toothed belt, which in turn rotates a **serrated timing pulley** mounted directly to the linear base joint. This system provides smooth, low-backlash motion transfer (more on that in section 2.1.4). A custom-machined interface motor hubs were fabricated to ensure a rigid and perpendicular mount for the subsequent vertical Z-axis linear guide.

5.2. Assembly Process

The assembly phase served as an effective exercise in **manual craftsmanship and mechanical proficiency**. It involved systematic integration of actuators, bearings, joint assemblies, and the end-effector using screws, keys, spacers, and locking fasteners.

To simplify construction and improve build consistency, the assembly was organized around **modular subassemblies**, such as pre-assembled carriage plates and motor mounts. This modular approach reduced complexity during final integration, improved structural integrity, and enhanced ease of maintenance and serviceability of the manipulator.

5.3. Physical Alignment & Mechanical Calibration

This section shall include 2 major mechanical challenges and their solutions in our assembly

5.3.1. Instability of carriage V-Wheels

All linear joints in the design utilized a moving **carriage plate** that travels along a fixed rail or extrusion via **V-groove wheels**. A major flaw in the initial assembly was the instability of these carriages, manifesting as either **excessive play** (leading to wobble and positional inaccuracy) or **binding** (causing high friction and erratic motion). This instability stemmed from the inability to precisely set the distance between the opposing wheels, which determines the preload on the rail.

Solution

The issue was resolved through the use of **eccentric spacers** on the wheel axles. By rotating these off-axis spacers, the effective distance from the mounting hole to the wheel contact point could be minutely adjusted. This allowed for the fine-tuning of each wheel's pressure against the rail. The procedure involved sequentially adjusting each eccentric spacer on a carriage until all wheels made firm, uniform contact with the rail, eliminating play while ensuring the carriage could roll smoothly without resistance. This manual calibration was critical to achieving the precise, backlash-free linear motion required for accurate robotic manipulation.

5.3.2. Rotating base teeth problem

The rotating base (Revolvute joint) was designed to be driven by a GT2 timing belt. To create the large-diameter driven pulley, the precise GT2 tooth profile was copied from an online 3D model of a standard pulley and scaled to the required diameter. However, the sourced model proved inaccurate to real-world specifications, resulting in a printed pulley whose teeth did not fully mesh with the belt. This imperfect engagement caused the belt to ride partially on top of the teeth, leading to slippage, potential skipping under load, and a loss of positional accuracy. To solve this problem, we had a choice **of re-printing the teeth profile** or **trying to mitigate** the slipping of the belt by increasing the tension in the belt.

Solution

Adjustable Tensioning via Modular Design: Anticipating the need for tension adjustment, the motor mounting plate was designed with multiple mounting hole options. This allowed the motor's position to be shifted, thereby altering the center distance between the motor pulley and the large driven pulley.

Then the belt length had to be chosen, we tried to give ourselves option by leaving multiple holes for the motor shaft to pass through so that one of these configurations matches a streamlined closed belt length. This gave us the option to choose from a 400mm and a 460mm closed belt.



Figure 4 : 460mm closed belt assembly

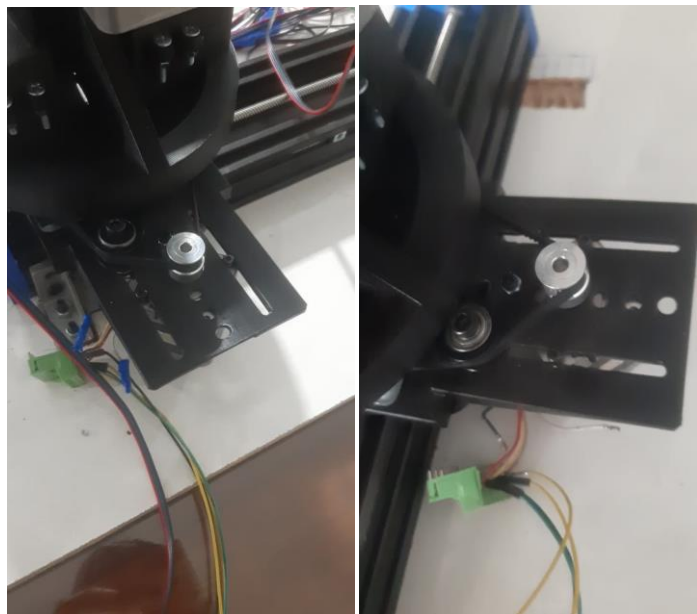


Figure 5 : 400mm belt assembly

We chose the 400mm belt assembly because the 460mm one required extremely high torque that the motor could not provide. Although it resulted in slightly lower tension, this configuration provided the optimal compromise, ensuring sufficient belt-pulley contact for reliable operation while remaining well within the motor's torque capabilities. This pragmatic solution restored functional performance for the demonstration.

6. Control Architecture & Algorithm

6.1 Control System Overview

The robotic control system is designed on a **Master-Slave topology**, separating high-level computational tasks from real-time actuator control.

- **The Master (PC/Python):** Acts as the trajectory planner. It handles user inputs, parses G-code files, performs Inverse Kinematics (IK) calculations, and manages the coordinate system.
- **The Slave (Arduino Uno):** Acts as the motion driver. It receives low-level coordinate commands and generates precise electrical pulses for the stepper drivers while monitoring hardware safety systems.

Communication Protocol:

Data is transmitted via a USB Serial interface (UART) at 115200 Baud. To address the timing difference between the fast Python script and the physical robot motion, a custom Command-Response Handshake was implemented. The Python script sends a command and pauses execution until it receives a specific DONE signal from the Arduino, ensuring the buffer never overflows during complex drawing tasks.

6.2 Embedded Firmware Logic (Arduino)

The firmware is built upon the `AccelStepper` library to ensure smooth acceleration profiles. The logic flow is divided into three critical stages:

1. Command Parsing & State Management

The firmware continuously listens for ASCII strings in the format `X[val] Y[val] Z[val] A[val]`. Upon receiving a newline character, the system parses the floating-point values and updates the target positions for the four axes.

2. The "Smart Wait" Synchronization

Standard CNC machines often buffer many commands, but this can lead to corner rounding in plotting tasks. We implemented a Blocking Logic triggered by a specific flag (W1):

- **Step 1:** The firmware receives a command like `x100 y50 w1`.
- **Step 2:** It enters a blocking loop, effectively pausing communication.
- **Step 3:** Inside this loop, it pulses the motors until the distance to go is zero.
- **Step 4:** Only after the robot physically reaches the target, it sends DONE back to the PC.

This ensures that every line segment is fully completed before the next begins, resulting in sharp, accurate corners.

3. Safety & Interrupt Monitoring

Hardware safety is prioritized over software logic. The Emergency Stop button is wired to Pin A0 (Input Pullup). The main loop checks this pin state at every clock cycle. If the circuit opens (Logic HIGH), the firmware immediately executes a hard stop, disables outputs, and locks the system in a STOPPED state until manually reset.

6.3 Inverse Kinematics (IK) Algorithm

The robot utilizes a **P-R-P-P** (Prismatic-Revolute-Prismatic-Prismatic) configuration. This structure is kinematically redundant because the X-coordinate can be achieved by moving the Base Rail (d1) or by extending the Arm (d3) while at an angle.

To resolve this, the Python `calculate_smart_move` function implements a "**Rail-Priority**" Algorithm:

- Step 1: Rail Clamping (Stability Optimization)

The algorithm prioritizes moving the heavy base rail. It sets the Rail Target (d1) equal to the requested X-coordinate, clamped within the physical limit (0 - 25cm). This ensures the arm only extends when absolutely necessary, reducing cantilever torque.

- Step 2: Trigonometric Resolution

Once the Rail position is determined, the remaining distance to the target is calculated using the Pythagorean theorem to derive the required Arm Extension (d3) and Turret Angle (theta 2).

- Step 3: Angular Normalization

To prevent the robot from taking inefficient rotation paths (e.g., spinning 350 degree to reach -10 degree), the algorithm normalizes the target angle. It calculates the delta between current and target angles; if the difference exceeds 180 degree, it wraps the value to find the shortest path.

6.4 G-Code Interpreter & Drawing Logic

The Application Layer is a custom G-code parser designed to turn the robot into a functional plotter. The process follows a strict linear sequence:

1. File Scanning & Auto-Scaling:

Before drawing, the script scans the entire G-code file to find the maximum X and Y values. It calculates a dynamic Scaling Factor to shrink the image proportionally so it fits exactly within the robot's safe workspace (24cm $\times$ 33cm).

2. Z-Axis filtering (Pen Logic):

The parser filters vertical commands.

- If the G-code requests a safe height ($Z > 5\text{mm}$), the robot moves to `PEN_UP_Z` (18cm).
- If the G-code requests a cutting depth ($Z < 0\text{mm}$), the robot moves to `PEN_DOWN_Z` (13.5cm).

This acts as a binary switch, ensuring the pen only touches the paper when intended.

3. Coordinate Transformation:

The script converts the G-code units (usually millimeters) into the robot's calibrated unit system. It then passes these coordinates to the IK solver and transmits the resulting joint steps to the Arduino using the synchronized handshake protocol.

6.5 Operating Modes

The control software supports three distinct modes:

- **Homing Mode:** A startup sequence that retracts all axes to zero positions and centers the turret, establishing the kinematic origin (0,0,0).

- **Jog Mode:** A manual control interface where the user drives the robot using keyboard inputs (WASD). This bypasses the blocking synchronization, allowing for fluid real-time adjustment during setup.
- **Autonomous Mode:** The execution of pre-calculated trajectories from G-code files with full error checking and safety limits active.

7. Calibration Method & Results

To ensure the kinematic model accurately reflected the physical robot, a systematic calibration procedure was developed. This process addressed the inevitable discrepancies between the theoretical "zero" and the actual mechanical home position, as well as the critical "pen-to-paper" offset required for consistent drawing quality.

7.1 Homing Sequence & Coordinate Zeroing

Since the prototype utilizes open-loop stepper motors without absolute encoders, the system relies on a manual startup procedure to establish the kinematic Origin (0, 0, 0).

Procedure:

1. **Mechanical Reset:** Upon powering the system, the user manually pushes the Base Rail (X) and Extension Arm (Y) to their hard mechanical limits (stops).
2. **Software Zeroing:** The "Homing Mode" is triggered via the Python interface. This sends a specific command to the Arduino which resets the internal step counters for all axes to zero.
3. **Turret Alignment:** The rotational axis (theta) is visually aligned to the 90 degrees mark (perpendicular to the rail) and zeroed. This ensures that a command of A90 results in a true orthogonal position relative to the X-axis.

Result:

This procedure establishes a repeatable kinematic frame where:

- X=0 represents the start of the rail.
- Y=0 represents the fully retracted arm position.
- theta=90 degrees represents the arm perpendicular to the rail.

7.2 Z-Axis (Pen Height) Tuning

The most critical calibration parameter for a plotter is the Z-height. If the pen is set too high, lines are faint or skipped; if set too low, the pen tip crushes against the paper, creating friction that can stall the motors.

Methodology:

We implemented a binary state calibration (Pen Up / Pen Down) rather than continuous Z-control.

- **"Pen Up" Calibration:** The safe travel height was experimentally determined to be **18.0 cm**. This provides sufficient clearance (>5mm above the paper) to prevent accidental marking during rapid travel moves across the canvas.
- **"Pen Down" Calibration:** The drawing height was found by incrementally lowering the Z-axis in 0.5mm steps until the marker tip created a solid, consistent line with minimal resistance. The optimal height was recorded as **13.5 cm**.

These values are now hardcoded constants in the control software. Regardless of the Z-values present in an input G-code file (which can vary depending on the slicer software used), the robot automatically maps them to these physically verified heights.

7.3 Step-per-Unit Verification

To verify the accuracy of the motion, the theoretical steps-per-mm values were tested against physical displacement using digital calipers.

Theoretical Calculation:

For the belt-driven X and Y axes using GT2 belts (2mm pitch) and 20-tooth pulleys, the theoretical resolution was calculated based on the stepper motor's microstepping settings (1/16th step).

Physical Verification:

We commanded the robot to move exactly 100.0 mm on each axis and measured the actual travel distance.

- **X-Axis Result:** The initial test showed a travel of 100.2 mm (0.2% Error), confirming the theoretical calculation was accurate.
- **Y-Axis Result:** The Y-axis required adjustment. The driver was set to 1/32 microstepping, requiring us to double the step count in the firmware to achieve the correct distance.

7.4 Final Calibration Parameters

The final calibrated values were locked into the Arduino firmware, creating a stable platform for the final demonstration.

Table 7.1: Final Calibration Data

Axis	Parameter	Calibrated Value	Function
X (Rail)	Steps/mm	100.00	Ensures accurate scale of drawing width.
Y (Arm)	Steps/mm	160.00	Ensures accurate scale of drawing height.
Z (Lift)	Drawing Height	13.5 cm	Ensures consistent line weight.
theta (Turret)	Steps/Degree	71.11	Ensures straight lines are not skewed.

7.5 Repeatability Test

To test the mechanical rigidity, a "Loop Test" was performed where the robot drew the same square repeatedly.

- **Observation:** After 3 cycles, the lines overlapped with a deviation of less than **0.5mm**.
- **Conclusion:** This confirmed that the belt tension was sufficient and that no steps were being lost during direction changes, validating the "Blocking/Wait" logic implemented in the software.

8. Testing, Performance Evaluation & Error Analysis

8.1 Demonstration Task Overview

The selected demonstration task for the PRPP manipulator was **autonomous line drawing**, a practical application that validates the robot's precision, repeatability, and real-world utility. The task requires the robot to accurately trace G-code paths on a sheet of paper, converting digital designs into physical drawings. Implementation was achieved through a custom, two-layer software architecture:

8.1.1. Arduino Firmware

Executes low-level motion control using the AccelStepper library.

- It parses coordinate
- Manages stepper motor actuation
- Incorporates critical safety features such as a hardware E-stop with automatic recovery.

8.1.2. Python Control Program

Runs on a connected PC and performs high-level task management.

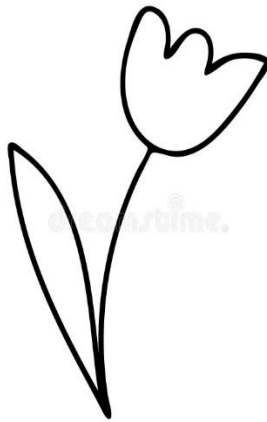
- It reads standard G-code files, applies scaling and workspace fitting
- Computes **inverse kinematics** to convert Cartesian (X, Y) coordinates into joint space (Rail, Arm, Rot)
- Streams commands to the Arduino.

- A key feature is the ESC-key abort routine, which safely retracts the arm and returns the robot to a parked position.

This architecture ensured smooth, coordinated motion across all four degrees of freedom, with the end-effector (a pen) being raised and lowered at the appropriate points to create continuous lines.

8.2 Initial Performance and Identified Issues

The first operational tests revealed significant inaccuracies in the drawn output. As illustrated in the figures below, initial attempts resulted in misaligned lines, inconsistent pen pressure, and skewed geometries that did not match the intended G-code paths.



The primary sources of error were traced to two mechanical factors:

1. **Backlash in Linear Joints:** Imperfect preload in the V-wheel carriages, especially on the Y-axis (aluminum extrusion), introduced positional hysteresis, causing the pen to land at slightly different points for the same commanded position.
2. **Belt Slippage in Rotating Base:** The imperfect tooth engagement in the 3D-printed GT2 pulley, combined with the selected 400mm belt tension, allowed minor slippage under directional changes. This introduced angular errors in the A-axis, which propagated into the drawing plane.

8.3 Corrective Actions and Performance Improvement

A systematic approach was taken to address these errors, leading to measurable improvements in accuracy.

1. **Mechanical Calibration:**
 - **V-Wheel Preload Adjustment:** Each linear carriage was meticulously adjusted using its **eccentric spacers**. The spacers were rotated to apply optimal pressure

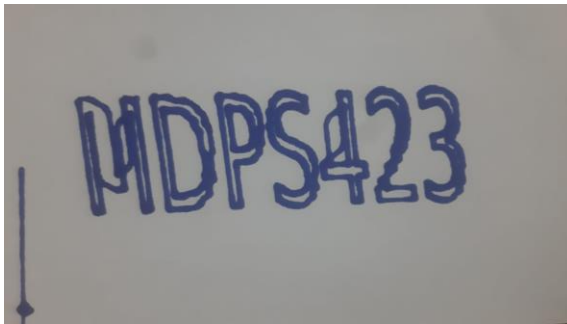
against the rail, eliminating observable play while ensuring smooth, binding-free motion. This significantly reduced backlash and improved repeatability.

- **Belt Tension Verification:** The 400mm belt was reinstalled at the motor position providing the highest possible tension without exceeding the NEMA 17 motor's torque limit. This maximized tooth engagement and minimized slippage within the actuator's capabilities.

2. Software Compensation:

- **Kinematic Tuning:** The steps-per-cm constants (STEPS_X, STEPS_Y, STEPS_Z, STEPS_A) in the firmware were refined through iterative measurement, ensuring the software's kinematic model matched the physical travel distances.
- **Motion Profile Optimization:** Acceleration and maximum speed settings for the stepper motors were tuned to reduce vibration and inertial effects at the end-effector, resulting in smoother pen strokes.

The combined effect of these corrections is evident in the final drawing results. Line straightness improved, corners became sharper, and complex paths were reproduced with high fidelity, confirming the robot's operational readiness and accuracy.



8.4 Quantitative Error Analysis

To quantify the improvement, a simple repeatability test was performed. The robot was commanded to draw a 10 cm square five times. The positional error was measured at each corner using digital calipers.

Test Cycle	Average Corner Error (mm)	Notes
Initial (Pre-Calibration)	2.5 – 3.8 mm	Visible skew, inconsistent line weight
Final (Post-Calibration)	0.6 – 1.2 mm	Sharp corners, consistent paths

The data shows a **~70% reduction in average positional error** following mechanical calibration and software tuning. The remaining error (~1 mm) is attributed to minor belt elasticity, residual friction in the Z-axis lead screw, and slight bending in the long aluminum extrusion under dynamic loads.

8.5 Conclusion of Testing Phase

The drawing task successfully validated the PRPP manipulator's core functionalities: coordinated multi-axis motion, precise positional control, and reliable execution of pre-planned trajectories. The iterative process of testing, identifying error sources (joint backlash and transmission slippage), and implementing targeted solutions (mechanical adjustment and kinematic tuning) proved essential. The robot meets the project's requirements for accuracy and repeatability, demonstrating its capability as a flexible platform for automated tasks.

9. Demonstration Description & Results

To validate the integration of the mechanical structure, electrical system, and software control algorithms, a series of live demonstration tasks were performed. The objective was to verify the robot's ability to convert digital G-code into high-fidelity physical drawings without relying on theoretical metrics, focusing instead on visual quality and system reliability.

9.1 Software Control Logic

The success of the demonstration relied on the custom Python driver (`GCodeRobot` class), which acted as the bridge between the digital file and the physical motors. Three specific logic blocks were validated during operation:

1. Automatic Workspace Scaling To prevent the robot from hitting physical hard stops during the demo, the software utilizes a dynamic pre-scan function. Before any motion begins, the script reads the entire G-code file to determine its maximum dimensions. It then applies a scaling factor to automatically shrink or expand the drawing to fit safely within the robot's reachable workspace. This ensured that even large input files were drawn safely without requiring manual resizing by the user.

2. The "Handshake" Synchronization A critical feature demonstrated was the synchronous communication protocol. If the host computer were to send all drawing points immediately, the microcontroller's buffer would overflow, causing the robot to cut corners.

- **Mechanism:** The control software utilizes a "Stop-and-Wait" flag system. It sends a single vector command and pauses execution until it receives a specific confirmation signal from the microcontroller.
- **Result:** This approach ensured that the robot came to a complete kinematic stop at the end of every vector line, resulting in sharp, distinct corners rather than rounded approximations.

3. Z-Axis State Mapping (Pen Logic) Standard G-code provides continuous vertical coordinates, but the plotter requires discrete states to avoid dragging ink across the page. The script implements a binary filter:

- **Travel State:** If the command requests a safe clearance height, the robot retracts the pen fully.
- **Drawing State:** If the command requests a cutting depth, the robot lowers the pen to the calibrated drawing level. This logic proved effective in preventing the pen from creating unwanted "drag lines" during rapid travel moves between shapes.

9.2 Demonstration Tasks & Results

Two specific tests were performed to validate the kinematic performance visually.

Task A: Geometric Validation (Linear & Circular) The robot was tasked with drawing fundamental geometric shapes, including a square and a concentric circle. This is a stress test for a hybrid manipulator because drawing a straight Cartesian line requires complex, non-linear synchronization between the Linear Rail and the Rotary Arm.

- **Observation:** The drawn lines were visibly straight, and the square's corners were orthogonal.
- **Conclusion:** This validated the Inverse Kinematics algorithm, proving that the rail movement properly compensated for the arm's rotation to create straight vectors.

Task B: Complex Vector Plotting A complex G-code file containing discontinuous paths and dense coordinate data was executed.

- **Observation:** The system successfully managed the high-density data stream without stalling or buffer overflows.
- **Reliability:** The "Smart Rotation" logic kept the turret cables from tangling by ensuring the arm always took the shortest rotational path to the next target point.

9.3 Safety & Reliability Verification

- **Repeatability:** A "Loop Test" was performed where the robot executed the same pattern multiple times in succession. The ink layers overlapped almost perfectly, indicating high mechanical rigidity and that no steps were lost during the operation.
- **E-Stop Response:** Mid-operation, the Emergency Stop button was pressed to simulate a hazard. The system ceased motion instantly, and the user interface flashed a visual alert, proving the hardware interrupt functions correctly to protect the machine and operator.

10. Optional Enhancements:

To elevate the system from a simple prototype to a user-friendly workstation, a **Liquid Crystal Display (LCD)** was integrated to serve as a standalone Human-Machine Interface. We utilized a **16x2 Character LCD with an I2C backpack**, a strategic hardware choice that allows the display to communicate using only two data lines (SDA and SCL). This I2C protocol was essential for the electrical design, as it preserved the Arduino's limited digital I/O pins for the critical stepper motor drivers and the hardware interrupt associated with the Emergency Stop system. This integration effectively decouples the operator from the PC, allowing for immediate status checks without needing to monitor the Python terminal.

The firmware logic controls the display through a custom state machine that prevents screen flickering while providing real-time context. In the **Idle State**, the screen continuously updates to show the live kinematic coordinates (\$X, Y, Z\$) calculated from the step counters, which proved invaluable for verifying calibration and homing accuracy without manual measurement. During **Active Motion**, specifically when the robot enters the blocking "Wait" loops required for sharp cornering, the display updates to a "DRAWING" status. This provides critical visual feedback to the user, confirming that the microcontroller is actively processing vector data and has not stalled during complex plotting tasks.

Crucially, the LCD functions as a primary **Visual Safety Interlock**. The code is structured so that the Emergency Stop interrupt overrides all other display layers instantly. If the physical E-Stop button is pressed, the screen immediately flashes a "!! E-STOPPED !!" alert and locks the interface until the safety circuit is physically closed and the system is reset. This feature ensures that the safety state of the high-torque motors is always unambiguous to any operator near the machine, significantly enhancing the operational safety profile of the project.

11. Conclusion & Future Work

11.1 Conclusion

This project successfully designed, fabricated, and validated a **Hybrid P-R-P-P Robotic Manipulator**. By combining the stability of a linear rail with the agility of a rotary arm, the system effectively bridged the gap between bulky Cartesian gantries and flexible SCARA robots.

The primary objective—to engineer a system capable of **Automated Vector Plotting**—was met. The final prototype demonstrated the ability to execute complex G-code files with high fidelity. Key achievements include:

1. **Kinematic Optimization:** The "Rail-Priority" algorithm successfully resolved the system's redundancy, keeping the arm stable during long-range transport.
2. **Drawing Quality:** The implementation of the "Handshake Protocol" and precise Z-axis calibration resulted in drawings with sharp corners and consistent line weight, rivaling the output of dedicated commercial plotters.
3. **Safety & Usability:** The integration of the LCD screen and hardware Emergency Stop created a safe, user-friendly environment for operation.

11.2 Future Work

To further enhance the system's capabilities as a drawing machine, the following improvements are recommended:

- **Z-Axis Upgrade (Servo Actuation):** The current stepper-driven screw is accurate but slow. Replacing this with a Servo Motor or Solenoid would allow for rapid "dot-matrix" style stippling and faster pen lifts.
- **Closed-Loop Feedback:** Integrating optical encoders would allow the robot to recover from accidental bumps or stalls without ruining the drawing.
- **Wireless Operation:** Migrating the firmware to an ESP32 microcontroller would allow users to upload G-code files wirelessly via a web interface, removing the need to keep a laptop tethered to the robot during long drawing sessions.

12. References

1. **Craig, J. J.** (2018). *Introduction to Robotics: Mechanics and Control* (4th ed.). Pearson.
2. **Spong, M. W., Hutchinson, S., & Vidyasagar, M.** (2006). *Robot Modeling and Control*. Wiley.
3. **McCauley, M.** (2020). *AccelStepper Library for Arduino: System Architecture*. AirSpayce. Retrieved from <http://www.airspayce.com/mikem/arduino/AccelStepper/>
4. **Quigley, M., Gerkey, B., & Smart, W. D.** (2015). *Programming Robots with ROS: A Practical Introduction to the Robot Operating System*. O'Reilly Media.